QUADCOPTER ARM OPEN LOOP CONTROL

MMAE 415 – Aerospace Laboratory II

Abstract

The goal of this lab experiment was to look at the open loop angle control of a single quadcopter arm using a brushless motor controlled by a pulse width modulation (PWM) signal. The objectives of the lab experiment were to vary the motor speed through a variable duty cycle input, determine the pitch angle of the arm from two accelerometer outputs, generate a static calibration map that relates the duty cycle to the pitch angle, and use an inverted calibration map to set the arm to specific angles. The static calibration map was generated using measured duty cycle and pitch angle data with a linear regression applied. The open loop response of the system was evaluated using a step change in the commanded angle from -60 degrees to -50 degrees, with the actual angle recorded by the accelerometer. The results showed that the quadcopter arm could be controlled through a change in the duty cycle and the angle could be approximated through the inverted calibration, but there were limits to the tracking accuracy and limits on the stability of the system for larger commanded angle changes.

Kaleb Martiny

3/25/2026

Table of Contents

Introduction

The control of a quadcopter is primarily determined by the relationship between the input of the motors and the thrust generated by the propellers. For a quadcopter to take off and accelerate vertically, the motors must spin faster, causing more lift to be generated. In this experiment, a single quadcopter arm and motor system was used to evaluate how the pulse width modulation (PWM) input to a motor affects the output arm position. The motor was driven by an electronic speed controller using a PWM signal generated in Simulink. The generated signal was then run through ControlDesk and manually adjusted before being sent to the speed controller. The angle of the quadcopter arm was estimated using dual outputs from an accelerometer attached to the arm, which recorded the gravitational acceleration vector components in the Y and Z axes. As this experiment focused on open loop behavior, the input to the motor was generated based on the static calibration and was not continuously corrected based on the measured output error. This contrasts with a closed loop system, where the output is continuously corrected based on its error from the input, resulting in a more accurate system response (Franklin, Powell, & Emami-Naeini, 2020).

There were four objectives that this lab sought to complete. The first was to control the speed of the brushless motor using a duty cycle signal generated by a PWM block in Simulink. The signal was sent through ControlDesk and was able to be adjusted manually to set the desired quadcopter arm angle. The second objective was to measure the angle of the quadcopter arm relative to the input PWM duty cycle. The angle was found using the Y and Z axes components from the accelerometer and was displayed in real time in ControlDesk. The third objective was to create and invert a static calibration map to be able to compute the necessary duty cycle to move the arm to a specific angle. The final objective was to record the time response of the quadcopter arm for a commanded step input from -60° to -50° and use the response as a measure of the open loop performance.

Relevant Equations

The PWM period that corresponds to a frequency of 50 Hz:

$$T_{PWM} = \frac{1}{f_{PWM}} = \frac{1}{50} = 0.02\ s \tag{1}$$

Where:

$T_{PWM} = PWM\ Period$
$f_{PWM} = Desired\ Frequency$

To calculate the angle of the quadcopter arm, both a sensitivity constant and offset constant were needed and were calculated using:

$$V_y = V_{y,offset} + S_y\left(\frac{a_y}{g}\right) \; and \; V_z = V_{z,offset} + S_z\left(\frac{a_z}{g}\right) \tag{2}$$

Where:

$V_y, V_z = Output\ Voltage$
$V_{y,offset}, V_{z,offset} = Voltage\ When\ the\ Directional\ Component\ and\ Gravity\ Equal$
$S_y, S_z = Y\ and\ Z\ Axes\ Sensitivities$
$a_y, a_z = Acceleration\ in\ Y\ and\ Z$
$g = Gravitational\ Acceleration$

To determine the sensitivities, the quadcopter arm was held at 0° and -90° and the voltages for both Y and Z were recorded. At 0° horizontal, the Z axis voltage, V_z, is at its maximum as the acceleration in the Z direction is 1g. The Y axis voltage, $V_{y,offset}$, is at its minimum as the acceleration in the Y direction is 0. At -90° vertical, the Z axis voltage, $V_{z,offset}$, is at its minimum as the acceleration in the Z axis is 0. The Y axis voltage, V_y, is at its maximum as the acceleration in the Y direction is 1g. With the voltages recorded at both horizontal and vertical, Equation 2 can be used to calculate the sensitivities.

Using the found and computed voltages and sensitivities, the angle can then be computed with:

$$\theta = -\sin^{-1}\left(\frac{V_y - V_{y,offset}}{S_y}\right) \; and \; \theta = -\cos^{-1}\left(\frac{V_z - V_{z,offset}}{S_z}\right) \tag{3}$$

Because of the small values of sin and cos near 0 and -90 degrees, a weighted average is taken using the output angles from Equation 3, found by:

$$\theta_{Avg} = \theta_y \cos^2(\theta_y) + \theta_z \sin^2(\theta_y) \tag{4}$$

To reduce measurement noise, a first order low pass filter was applied to incoming signal using a transfer function with a cutoff frequency of 5 Hz. The transfer function time constant was calculated using:

$$f_c = \frac{1}{2\pi\tau} \tag{5}$$

Where:

$f_c = Cutoff\ Frequency$
$\tau = Time\ Constant$

The resulting time constant from Equation 5, calculated as $\tau = 0.03183$, was placed into the following transfer function:

$$\frac{1}{\tau s + 1} \tag{6}$$

Experimental Setup

The setup for the experiment consisted of one arm of a quadcopter fitted with a brushless motor, electronic speed controller, propeller, and accelerometer. The arm was attached to a pivot at the opposite end of the motor to allow the arm to rotate from about -90° to 90° as seen in Figure 2. MATLAB, Simulink, dSpace ControlDesk and MicroLabBox, a Siglent SDS 1102CML+ oscilloscope, a BK Precision 1760A power supply, a Global Industries PB-505 bread board, and an external battery were the remaining equipment used in the experiment. During the initial setup of the experiment, the propeller was not attached and the battery to power the motor was not connected for safety reasons. The accelerometer was powered with 3.3 VDC and the two outputs for Y and Z were checked on the oscilloscope to ensure the correct response to movement of the arm. The two output voltages from the accelerometer were then acquired through ADC channels 1 and 2 and displayed in ControlDesk.

A Simulink model was created to convert the output accelerometer voltages into a pitch angle reading and to apply the first order low pass filter. The weighted average angle found from Equation 4 was displayed in real time in ControlDesk as well. To control the motor, a PWM block was inserted into the Simulink model with a period of 0.02 seconds, as found in Equation 1. The output PWM signal was run through the dSpace MicroLabBox and connected to the electronic speed controller through the breadboard, and a numeric input was added in ControlDesk to allow for the manual control of the duty cycle sent to the motor. The range for the duty cycle was set between 0.01 and 0.012 and was allowed to increase by increments of 0.001. The minimum and maximum duty cycles were found to give the full range of control of the motor.

After the circuit setup was verified, the propeller was installed and a throttle calibration was done. The minimum duty cycle, DTC_{min}, was the duty cycle where the motor just started to spin. The maximum duty cycle, DTC_{max}, was the duty cycle where the speed of the motor no longer increased. Between DTC_{min} and DTC_{max}, eleven duty cycle and angle points were recorded to produce a static map of θ vs DTC. This was plotted and then inverted with a linear regression applied in order to get an estimate for the duty cycle when an angle is input. This was implemented into the Simulink model and set up in ControlDesk so that either a constant duty cycle or an angle derived duty cycle could be used as the PWM input. The system was then tested by applying a step command from -60° to -50° and recording the response of the commanded angle, measured angle, and duty cycle, all as functions of time.

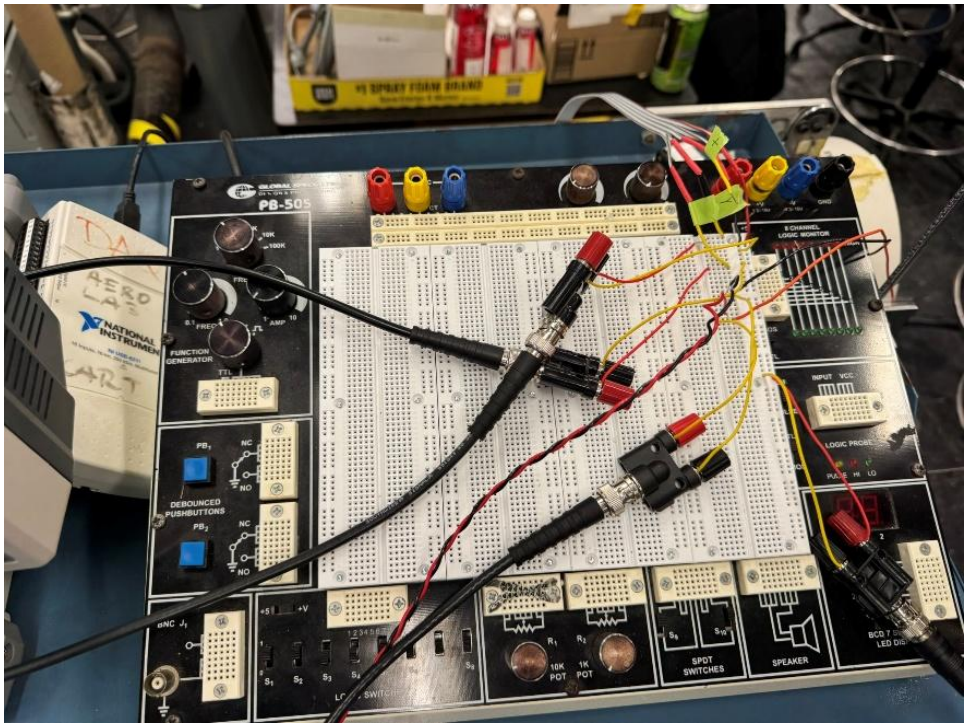


Figure 1: Circuit setup for PWM control and accelerometer reading.

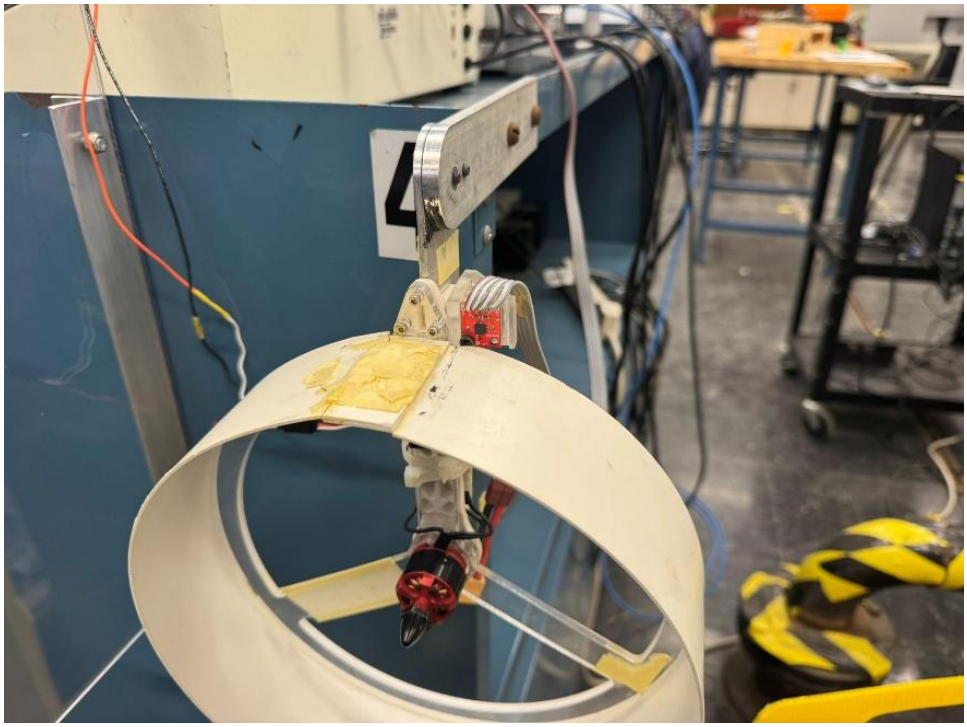


Figure 2: Quadcopter arm with motor and accelerometer.

Results

The minimum recorded duty cycle, DTC_{min}, was found to be 0.058, while DTC_{max} was found to be 0.089. To find the static calibration map, the following data was collected:

Duty Cycle	0.058	0.0611	0.0642	0.0673	0.0704	0.0735	0.0766	0.0797	0.0828	0.0859	0.0888
Angle	-95.6	-94.4	-88.3	-73.4	-74.5	-68.5	-62.1	-54.2	-46.3	-38.4	-21.6

Table 1: Duty cycle and measured angle recorded from ControlDesk for static calibration.

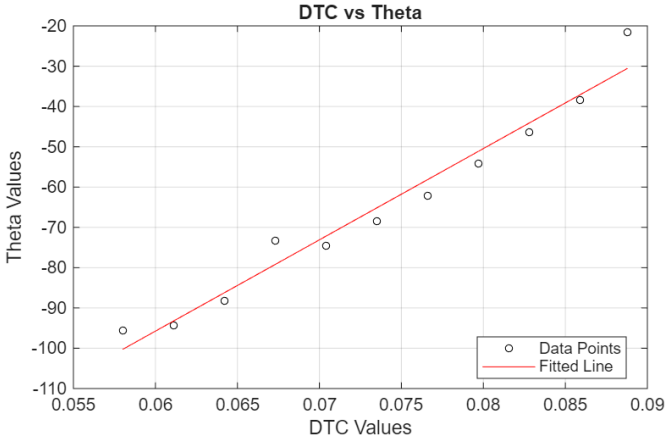


Figure 3: DTC vs Theta with a linear regression from the data in Table 1.

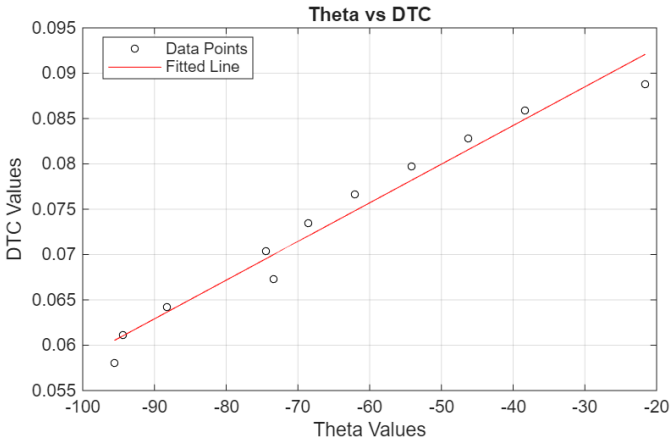


Figure 4: Theta vs DTC with a linear regression. This is Figure 3 inverted to allow the determination of a duty cycle value based on an input angle.

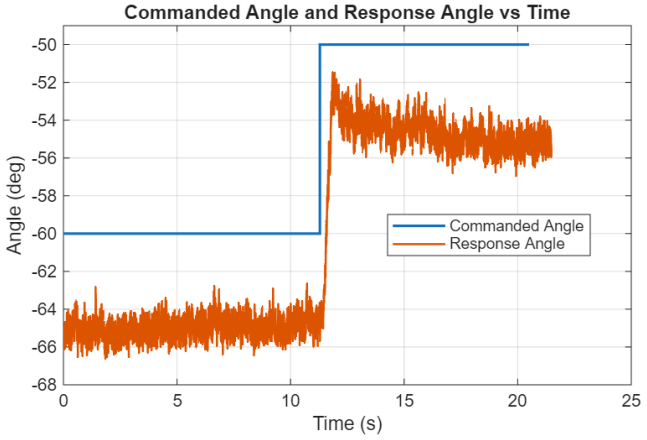


Figure 5: Recorded data for the commanded step input from -60° to -50°. An approximately 5 degrees offset can be seen across the entire test.

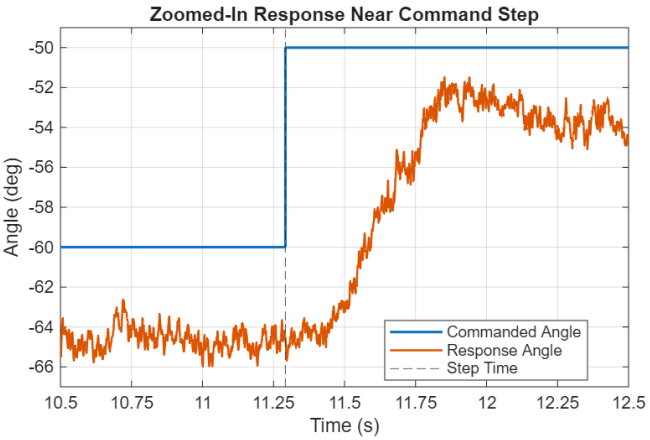


Figure 6: Response of the quadcopter arm right around the step input. The measured angle is seen to be lagging the commanded input as the motor takes time to speed up and propeller thrust to increase.

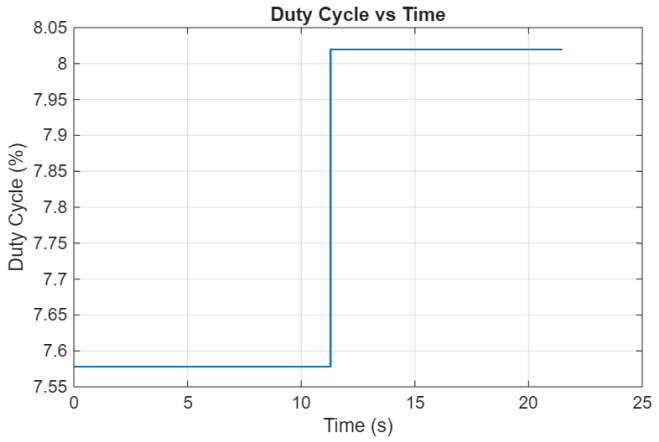


Figure 7: Duty cycle input to the motor over time with the step input.

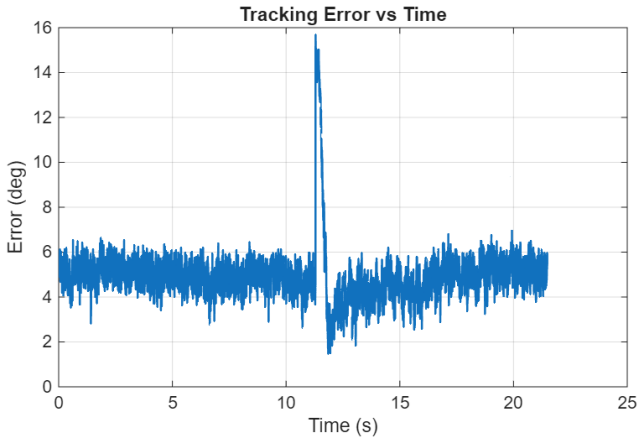


Figure 8: The error between the commanded angle and the measured angle in ControlDesk. The error stays consistently around 5 degrees, with a spike in error occurring at the commanded angle step input.

Figures 3 through 8 show that the static calibration of the quadcopter arm resulted in a usable relationship between a input duty cycle and the resulting arm angle, and that inverting this relationship allowed an input angle to give a corresponding duty cycle. The measured response of the system generally followed the commanded input from -60° to -50°, however there was a constant offset of about 5° over the course of the recorded data. Looking at the step change, a short lag is visible right after the command from -60° to -50°, and the error in Figure 8 showed an increase during this lag before returning to the pre-step level.

The Simulink model used in the experiment incorporates the reading of the accelerometers data from the Y and Z axes and subsequent conversion to angles, as well as the generation of the PWM duty cycle used to control the motor speed. The full model can be found in Appendix B.

A MATLAB script was used to generate the static map, as well as Figures 3 through 8. The data used was exported from ControlDesk as a .mat file and imported into MATLAB for processing and analysis. The full script used can be found in Appendix A.

Discussion of Results

The results show that the quadcopter arm changed angles in a predictable way in response to an input duty cycle, allowing the generation of a static calibration map based on several duty cycle and angle measurements across a broad range of angles. Although a linear regression was used to create the calibration map and its corresponding inverse, the data from Table 1 and the results in Figures 3 and 4 show that there was some inconsistency in the recorded data. Generally, the linear regression fit the data well, with only 3 of the 11 datapoints being outliers. These outliers indicate that there were some inconsistencies within the system, with possible sources being

friction at the pivot, variations in thrust, inaccurate accelerometer readings, and small variations in power delivery by the battery. Friction at the pivot could cause the arm to become stuck at a lower or higher position than expected. Variations in thrust could cause the arm to be unstable at a given duty cycle or behave in a nonlinear way. Inaccurate accelerometer readings could stem from the average used in Equation 4 and would alter the measured angle for a given duty cycle. Small variations in power delivery could cause the system to behave similarly to that of variations in thrust, mainly instability and nonlinearity.

The time response plots show that the main limitation in using a purely open loop approach, rather than closed loop. In Figures 5 and 6, the measured angle by the accelerometer followed the commanded angle change from -60° to -50° correctly, but it did not match the commanded angle exactly, displaying a consistent 5° offset as seen in Figure 8. This consistent offset indicates that using only static calibration is flawed if capturing a systems true behavior is desired. Any inconsistency in the data used to create the map are held throughout the response of the system. Because of the 3 inconsistent datapoints, the linear regression did not perfectly match the data and this can at least partially be seen in the 5° offset. Because the input duty cycle was purely determined by the static map and not corrected based on the measured output, any modeling errors remained in the final response. That said, the lag seen right after the step input is expected, as the motor needs time to speed up and generate the additional thrust needed to change the angle of the arm.

Overall, the results here show that using open loop control is able to move the quadcopter arm and provide general trends, but is not accurate enough for full control as there is no way to correct for errors introduced into the system or implicit to the system. For greater accuracy, a closed loop PID controller could be used to correct the duty cycle based on the difference between the commanded and measured angle.

Conclusion

This experiment successfully demonstrated the open loop control of a quadcopter arm using a PWM signal. The signal was sent from Simulink through a dSpace MicroLabBox to the electronic speed controller for a brushless motor with a propeller attached. A static calibration was performed to create a linear relationship between duty cycle and quadcopter arm angle. The inverse of this relationship was taken to allow for the estimation of required duty cycle for a commanded angle input. This allowed for the control of the quadcopter arm and was successfully demonstrated by commanding the arm to go from -60° to -50° via a step change.

Despite this success, the open loop response also showed its limitations. The measured angle was consistently off the commanded by about 5°, and a lag was observed immediately after the step change was issued, however this is expected as the motor and propeller need time to generate the

necessary thrust to change the angle of the arm. The results show that use of a static calibration is useful for approximate control, but that a closed loop system with PID control would be needed for more accurate tracking and to reduce steady state error. In future experiments, a more thorough construction of the static map should be taken. While the use of eleven datapoints was sufficient for general trends, increasing the number of points taken would increase the accuracy of the linear regression. It would also be worthwhile to record data points for the motor decreasing in angle as well, not just increasing. This would give a more comprehensive look at the duty cycle and angle relationship.

Appendix A

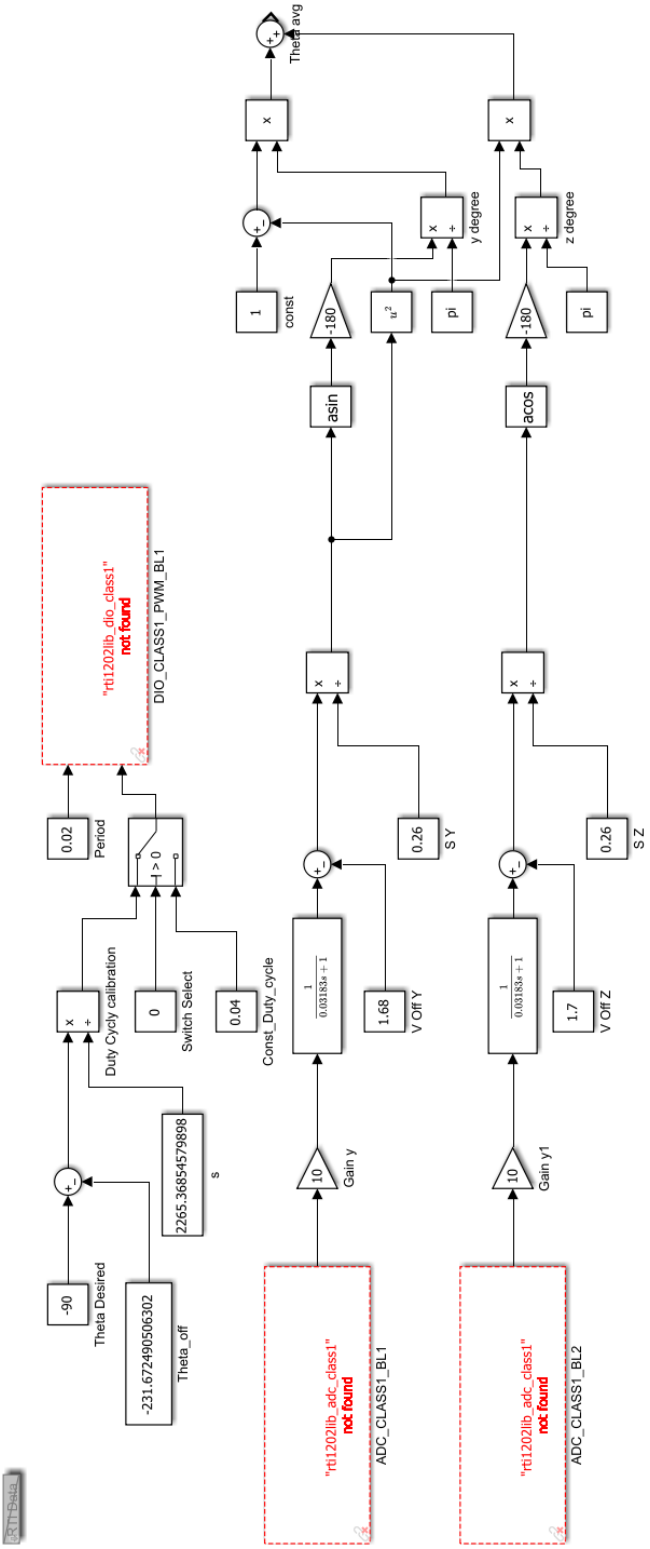
```
clc; clear;

data = load("rec1_003.mat");

DTC = [0.058 0.0611 0.0642 0.0673 0.0704 0.0735 0.0766 0.0797 0.0828 0.0859
0.0888];
theta = [-95.6 -94.4 -88.3 -73.4 -74.5 -68.5 -62.1 -54.2 -46.3 -38.4 -21.6];

p = polyfit(DTC,theta,1);
var = polyval(p,DTC);

figure(1)
plot(DTC, theta, 'o', 'MarkerSize', 4, 'Color', 'black');
hold on
grid on
plot(DTC, var, 'Color', 'red');
xlabel('DTC Values');
ylabel('Theta Values');
title('DTC vs Theta');
legend('Data Points', 'Fitted Line', 'Location', 'best');
hold off

p_inv = polyfit(theta, DTC, 1);
var_inv = polyval(p_inv, theta);

figure(2)
plot(theta, DTC, 'o', 'MarkerSize', 4, 'Color', 'black');
hold on
grid on
plot(theta, var_inv, 'Color', 'red');
xlabel('Theta Values');
ylabel('DTC Values');
title('Theta vs DTC');
legend('Data Points', 'Fitted Line', 'Location', 'best');
hold off


S = data;
R = S.rec1_003;

% Extract signals
```

```
t = R.X(1).Data;            % continuous time vector
t_cmd = R.X(2).Data;        % commanded-angle event times

duty = R.Y(1).Data;         % Model Root/Switch
theta_resp = R.Y(2).Data;   % Model Root/Theta avg
theta_cmd = R.Y(3).Data;    % Model Root/Theta Desired

% Force column vectors
t = t(:);
t_cmd = t_cmd(:);
duty = duty(:);
theta_resp = theta_resp(:);
theta_cmd = theta_cmd(:);

% Clean commanded-angle time vector
keep = t_cmd >= 0;
t_cmd = t_cmd(keep);
theta_cmd = theta_cmd(keep);

% Make sure commanded signal starts at t = 0
if isempty(t_cmd) || t_cmd(1) > 0
    t_cmd = [0; t_cmd];
    theta_cmd = [theta_cmd(1); theta_cmd];
end

% Duty cycle in percent, if desired for plotting
duty_percent = 100*duty;

%% Plot 1: Commanded angle and response angle versus time
figure;
stairs(t_cmd, theta_cmd, 'LineWidth', 1.5); hold on;
plot(t, theta_resp, 'LineWidth', 1.2);
grid on;
xlabel('Time (s)');
ylabel('Angle (deg)');
ylim([-68 -49]);
title('Commanded Angle and Response Angle vs Time');
legend('Commanded Angle', 'Response Angle', 'Location', 'best');

%% Plot 2: Duty cycle versus time
figure;
plot(t, duty_percent, 'LineWidth', 1.2);
grid on;
xlabel('Time (s)');
```

```
ylabel('Duty Cycle (%)');
title('Duty Cycle vs Time');

%% Optional Plot 3: Tracking error versus time
% Build a piecewise-constant commanded signal on the continuous time base
theta_cmd_full = interp1(t_cmd, theta_cmd, t, 'previous', 'extrap');
tracking_error = theta_cmd_full - theta_resp;

figure;
plot(t, tracking_error, 'LineWidth', 1.2);
grid on;
xlabel('Time (s)');
ylabel('Error (deg)');
title('Tracking Error vs Time');

%% Optional Plot 4: Zoomed-in transient response around the command step

% Find the first actual step change in commanded angle
idx_step = find(diff(theta_cmd) ~= 0, 1, 'first');

if ~isempty(idx_step)
    t_step = t_cmd(idx_step + 1);

    % Choose zoom window around the step
    t_before = 1.5;   % seconds before step
    t_after  = 4.0;   % seconds after step

    t_min = t_step - t_before;
    t_max = t_step + t_after;

    % Indices for continuous signals
    zoom_idx = (t >= t_min) & (t <= t_max);

    % Command signal over same window
    theta_cmd_zoom = interp1(t_cmd, theta_cmd, t(zoom_idx), 'previous',
'extrap');

    figure;
    stairs(t(zoom_idx), theta_cmd_zoom, 'LineWidth', 1.5); hold on;
    plot(t(zoom_idx), theta_resp(zoom_idx), 'LineWidth', 1.2);
    xline(t_step, '--');
    grid on;
    xlabel('Time (s)');
    xlim([10.5 12.5]);
```

```
    xticks(linspace(10.5, 12.5, 9));
    ylabel('Angle (deg)');
    ylim([-67 -49])
    title('Zoomed-In Response Near Command Step');
    legend('Commanded Angle', 'Response Angle', 'Step Time', 'Location',
'best');
 else
    warning('No step change was found in the commanded angle signal.');
 end
```

Appendix B


Theta avg
x
x
y degree
z degree
1
const
-180
pi
-180
pi
asin
acos
"rti1202lib_dio_class1"
not found
DIO_CLASS1_PWM_BL1
0.02
Period
I > 0
0.26
S Y
0.26
S Z
Duty Cycly calibration
0
Switch Select
0.04
Const_Duty_cycle
1.68
V Off Y
1.7
V Off Z
2265.36854579898
s
10
Gain y
10
Gain y1
-90
Theta Desired
-231.672490506302
Theta_off
"rti1202lib_adc_class1"
not found
ADC_CLASS1_BL1
"rti1202lib_adc_class1"
not found
ADC_CLASS1_BL2

References

Franklin, G. F., Powell, D. J., & Emami-Naeini, A. (2020). *Feedback Control of Dynamic Systems*. New York: Pearson.